

Lecture 20

Advantages of Linked Lists

- A linked list is dynamic data structure. Therefore, the primary advantage of linked lists over arrays is that linked list can grow or shrink in size during the execution of program.
- Linked list does not waste memory space. It uses the memory that is just needed for the list at any point of time.
- Linked lists provide flexibility is allowing the items to be arranged efficiently. It is easier to insert or delete items by rearranging the the links.

Limitations of Linked List

- Inorder to access any arbitrary item from linked list it is little cumbersome and time consuming.
- Linked list uses more storage than an array with the same number of items. This is because each item has an additional link field.

Type of Linked Lists

- Linear linked lists
- Circular linked lists
- Doubly or Two-way linked lists
- Circular doubly linked lists

Pointers Revisited

A. Initialization

```
int a;
```

```
int *p;
```

```
p = &a;
```

B. Assignment of Address

```
int a = 100, b = 200;
```

```
int *p = &a, *q = &b;
```

```
p = q;
```

continue...

C. Assignment of value

```
int a = 100, b = 200;
```

```
int *p = &a, *q = &b;
```

```
*p = *q;
```

D. NULL pointers

A special constant known as NULL pointer which initialize pointers that points to nothing.

```
p = 0 or p = NULL;
```

```
q = 0 or q = NULL;
```

continue...

We know that a pointer must be initialized by assigning a memory address before using it. There are two ways of assigning memory address to a pointer.

1. Assigning an existing variable address (static assignment)

```
prt = &count;
```

2. Using a memory allocation function (dynamic assignment)

```
ptr = (int *) malloc(sizeof(int));
```

Creating a linked list

We can treat a linked list as an abstract data type and perform the following basic operations:

1. Creating a linked list
2. Traversing the liked list
3. Counting the items in the linked list
4. Printing the linked list
5. Looking up an item for editing or printing
6. Inserting an item
7. Deleting an item
8. Concatenating two lists

continue...

The **head** is a pointer/reference that stores the address of the first node in the linked list. We use head to create and access anonymous nodes in the linked list.

```
struct linked_list{  
    int number;  
    struct linked_list *next;  
};  
typedef struct linked_list node;  
node* head;  
head = (node*) malloc(sizeof(node));
```

continue...

The following statement obtains a piece of memory that is sufficient to store a node and assigns its address to the pointer variable head. Head indicates the beginning of the linked list.

```
head = (node*)malloc(sizeof(node));
```

The following statements store values in the member fields:

```
head -> number = 10;
```

```
head -> next = NULL;
```

The second node can be added as follows:

```
head -> next = (node*)malloc(sizeof(node));
```

```
head -> next -> number = 20;
```

```
head -> next -> next = NULL;
```

Although this process becomes clumsy if nodes are more than two. The above process may be implemented using both recursion and iteration techniques.

Create Linked List

```
struct linked_list{  
    int number;  
    struct linked_list *next;  
}  
  
typedef struct linked_list node;
```

continue...

```
void create(node* currentNode){
    printf("Input a number");
    printf("type -999 to end the linked list);
    scanf("%d", &currentNode -> number);
    if(currentNode -> number == -999){
        currentNode -> next = NULL;
    } else {
        currentNode -> next = (node *) malloc(sizeof(node));
        create(currentNode -> next);
    }
    return;
}
```

Print Linked List

```
print(node *currentNode):  
    if currentNode -> next != NULL:  
        print number present in current node  
        if currentNode -> next -> next == NULL  
            print currentNode -> next -> number;  
        print(currentNode -> next);  
return
```

Count number of node present in linked list

```
int count (node* currentNode):  
    if currentNode -> next == NULL  
        return 0;  
    else:  
        return 1 + count(currentNode -> next);
```